H5: Loops deel 2

Labo 1 · week 6 · for en de Math-klasse

SEMESTER 1 · BASIS PROGRAMMEREN

Vandaag

1. **Opwarmen**: een `while` die telt
2. `for`: de teller-loop in één regel
3. **Valkuilen** met `for`
4. **De `Math`-klasse**: machten, wortels, π en afronden
5. **Aan de slag** met de for-oefeningen



Vandaag: `for`. Voor wie? Voor jullie.

Opwarmen

Wat is de output? 🤔

```
1  int evenNumber = 0;
2  while (evenNumber <= 10)
3  {
4      Console.WriteLine(evenNumber);
5      evenNumber = evenNumber + 2;
6  }
```

```
0
2
4
6
8
10
```

Een teller op **drie plaatsen**: beginwaarde, test en verhoging.

for

Alles op één regel

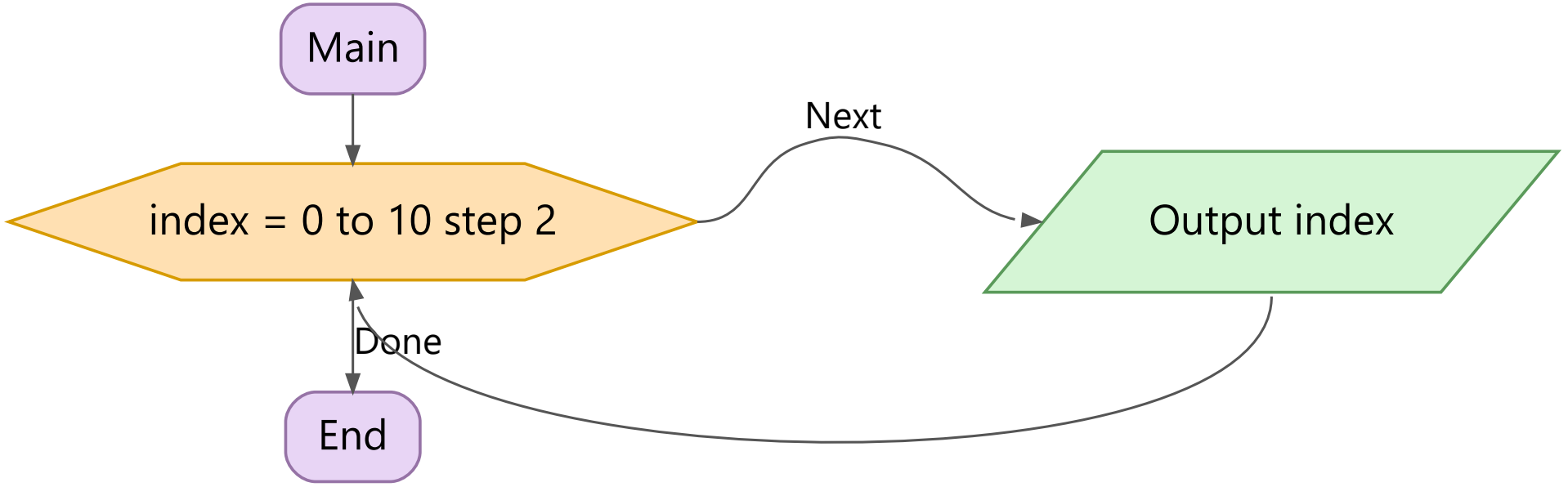
```
1 for (setup; test; update)
2 {
3     // de "body" van de loop
4 }
```

- **setup**: wordt **één keer** uitgevoerd, voor de loop start. Meestal maak je hier de **loopvariabele** aan.
- **test**: wordt **voor elke ronde** gecontroleerd. Is ze `false`, dan stopt de loop.
- **update**: wordt **na elke ronde** uitgevoerd. Meestal verhoog of verlaag je hier de loopvariabele.



Drie stukken tussen de haakjes, gescheiden door puntkomma's. Een sandwich van code.

In Flowgorithm




```
1  for (int index = 0; index <= 10; index += 2)
2  {
3      Console.WriteLine(index);
4  }
```

Next en Done: de enige twee knoppen die je op maandagochtend nodig hebt.

Nieuw: `+=`

```
1  counter += 2;    // betekent: counter = counter + 2;
```

Kort	Lang
`counter++`	`counter = counter + 1`
`counter--`	`counter = counter - 1`
`counter += 2`	`counter = counter + 2`
`counter -= 3`	`counter = counter - 3`
`counter *= 2`	`counter = counter * 2`

 `counter += 2`: *korter typen, langere koffiepauze.*

In welke volgorde?

```
1 for (int index = 1; index <= 3; index++)
2 {
3     Console.WriteLine(index);
4 }
```

Stap	`index`	Test `index <= 3`	Body
setup	1		
ronde 1	1	`true`	toont 1
ronde 2	2	`true`	toont 2
ronde 3	3	`true`	toont 3
einde	4	`false`	

Setup, test, body, update. Het klinkt als een fitnessschema, maar dan voor je processor.

Wat is de output? 🤔

In de body staat telkens `Console.Write($"{index} ");`

Loop	Output
`for (int index = 0; index < 5; index++)`	`0 1 2 3 4`
`for (int index = 1; index <= 5; index++)`	`1 2 3 4 5`
`for (int index = 10; index > 0; index -= 3)`	`10 7 4 1`
`for (int index = 10; index >= 1; index--)`	`10 9 8 7 6 5 4 3 2 1`



10, 7, 4, 1: aftellen in stappen van 3. Zo vertrekt de raket sneller.

`for` en `while`: hetzelfde

`while`

```
1  int index = 0;
2  while (index <= 10)
3  {
4      Console.WriteLine(index);
5      index += 2;
6  }
```

`for`

```
1  for (int index = 0; index <= 10; index += 2)
2  {
3      Console.WriteLine(index);
4  }
```

- `for`: als je vooraf weet **hoe vaak** (een counted loop)
- `while` of `do while`: als je het vooraf **niet** weet (een sentinel loop)

😈 *"Snap je het niet? Lees dit hoofdstuk dan opnieuw." Wat voor loop is dat?*

`for` met een `if`

De veelvouden van 7 tot 50:

```
1  for (int index = 1; index <= 50; index++)
2  {
3      if (index % 7 == 0)
4      {
5          Console.Write($"{index} ");
6      }
7  }
```

```
7 14 21 28 35 42 49
```

Veelvouden van 7. James Bond zou trots zijn.

`for` met een som

De som van 1 tot 100:

```
1  int sum = 0;
2  for (int index = 1; index <= 100; index++)
3  {
4      sum += index;
5  }
6  Console.WriteLine(sum);
```

```
5050
```

Gauss had geen for-loop nodig. Gauss wás een for-loop.

Valkuilen

Drie klassiekers

1. De loopvariabele na de loop gebruiken

```
1  for (int index = 1; index <= 3; index++)
2  {
3  }
4  Console.WriteLine(index);  // ❌ CS0103
```

2. Een puntkomma achter de haakjes

```
1  for (int index = 1; index <= 3; index++);  // ⚠️ CS0642
```

3. Eentje te veel of te weinig: `< 10` of `<= 10`? Begint het bij 0 of bij 1?



Er zijn twee moeilijke dingen in programmeren: namen kiezen, cache-invalidatie en off-by-one-fouten.

De `Math`-klasse

Wiskunde, kant-en-klaar

De klasse `Math` bevat methoden voor typische wiskundige bewerkingen: machten, wortels, afronden, ...

Het volume van een kubus met zijde 4:

```
1  double side = 4;
2  double volumeByHand = side * side * side;  // 64
3  double volume = Math.Pow(side, 3);          // 64
```

`Math.Pow(grondtal, exponent)` is leesbaarder, en makkelijk aan te passen naar een andere macht.

> **TIP**
>
> Typ in Visual Studio `Math.` en je ziet alles wat de klasse kan. Typ `Math.Pow(` en je krijgt de uitleg erbij.

De `Math`-klasse: wiskunde, zonder dat je het zelf moet kunnen. Bijna te mooi.

Machten en wortels

```
1  Console.WriteLine(Math.Pow(2, 10));  // 1024
2  Console.WriteLine(Math.Sqrt(81));    // 9
```

> ⚠ **WAARSCHUWING**
>
> `Math.Pow` en `Math.Sqrt` geven een **`double`** terug. `int power = Math.Pow(2, 3);` geeft fout **CS0266**. Gebruik dus `double power = Math.Pow(2, 3);`.

Samen met een `for`:

```
1  for (int index = 0; index <= 10; index++)
2  {
3      Console.Write($"{Math.Pow(2, index)} ");
4  }
```

```
1 2 4 8 16 32 64 128 256 512 1024
```

`Math.Pow(2, 10)` is 1024. Ongeveer het aantal keer dat je "nog één oefening" zegt.

π

`Math.PI` is geen methode, maar een vaste **waarde**: zonder haakjes.

```
1  double radius = 5.5;
2  double circumference = Math.PI * 2 * radius;
3  Console.WriteLine(circumference);
```

```
34,55751918948772
```

Met Belgische taalinstellingen toont de console een **komma**. In je code schrijf je altijd een **punt**: `5.5`.

😈 `Math.PI`: *taart voor programmeurs.*

Afronden

```
1  Console.WriteLine(Math.Round(12.6));          // 13
2  Console.WriteLine(Math.Round(3.14159, 2));    // 3,14
3  Console.WriteLine(Math.Floor(12.6));          // 12
4  Console.WriteLine(Math.Ceiling(12.2));        // 13
```

- `Math.Round`: naar het **dichtstbijzijnde** getal, eventueel met een aantal cijfers na de komma
- `Math.Floor`: altijd naar **beneden**
- `Math.Ceiling`: altijd naar **boven**

 *`Math.Ceiling`: altijd naar boven afronden. Zoals de prijs van een pak friet.*

Nog handige methoden

Methode	Voorbeeld	Resultaat
`Math.Max`	`Math.Max(7, 3)`	`7`
`Math.Min`	`Math.Min(7, 3)`	`3`
`Math.Abs`	`Math.Abs(-5)`	`5`
`Math.Sqrt`	`Math.Sqrt(81)`	`9`

Wie nieuwsgierig is: `Math.` typen en ontdekken. Er staat nog veel meer in, zoals sinus en cosinus.

 *`Math.Abs(-5)` is 5. Positief denken, in één methode.*

Samengevat

Wat	Hoe
tellen	`for (int index = 0; index < 10; index++) { }`
stappen	`counter += 2`, `counter--`, `counter -= 3`
macht en wortel	`Math.Pow(2, 3)`, `Math.Sqrt(81)` (een `double`!)
π	`Math.PI` (zonder haakjes)
afronden	`Math.Round`, `Math.Floor`, `Math.Ceiling`

 `for (int index = 1; index <= 1; index++) { Console.WriteLine("Bedankt!"); }` *Eén keer volstaat.*

Aan de slag 💻

Maak verder in de klasse `Loops`:

1. **Minus100Plus100**: toon de getallen van -100 tot 100, en daarna enkel de even getallen
2. **AMultiplicationTable**: toon de tafel van vermenigvuldiging van een getal
3. **Multiples6And8**: toon de getallen tot 100 die een veelvoud zijn van 6 en van 8
4. **Fibonacci**: toon de eerste getallen van de rij van Fibonacci
5. **PrimeNumberChecker**: is een getal een priemgetal?

👉 Klaar? Dan **PrimeNumberGenerator**. Week 5 nog niet af? Werk die oefeningen eerst af.

👹 `for (int index = 1; index <= 5; index++) { SolveExercise(index); }`